LeetCode — Maximum Alternating Score

Language: Java

Question

You are given an integer array **nums**. You may rearrange its elements in any order. The alternating score is arr[0]² – arr[1]² + arr[2]² – arr[3]² + ... Return the maximum possible alternating score after rearranging the elements.

Example 1

Input: nums = [1, 2, 3]
Output: 12

Explanation: Rearrange the array as [2, 1, 3]. The score is $2^2 – 1^2 + 3^2 = 4 – 1 + 9 = 12$.

Example 2

Input: nums = [1, -1, 2, -2, 3, -3]
Output: 16

Explanation: One optimal arrangement is [-3, -1, -2, 1, 3, 2]. Its score is 9 – 1 + 4 – 1 + 9 – 4 = 16.

Correct Java Solution

```
import java.util.Arrays;

class Solution {
    public long maxAlternatingSum(int[] nums) {
        long[] squares = new long[nums.length];

        for (int i = 0; i < nums.length; i++) {
            squares[i] = (long) nums[i] * nums[i];
        }

        Arrays.sort(squares);

        long score = 0;
        int n = squares.length;
        int negativeCount = n / 2;

        for (int i = 0; i < negativeCount; i++) {
            score -= squares[i];
        }

        for (int i = negativeCount; i < n; i++) {
            score += squares[i];
        }

        return score;
    }
}
```

Simple Logic

1. Square every number. Negative numbers also become non-negative after squaring.
2. Sort the squared values from smallest to largest.
3. Subtract the smallest floor(n/2) squared values.
4. Add all remaining squared values. This places the largest values in positive positions.

Complexity

Time: O(n log n), due to sorting.
Space: O(n), for the squared-values array.